

Sobel 算子步长 2 计算案例

<https://gemini.google.com/app/32abb557aeb5936b>

User prompt: sobel kernel 步长 2 计算案例

Response: 在图像处理和卷积神经网络 (CNN) 中, **Sobel 算子**通常用于边缘检测。当卷积核的步长 (**Stride**) 设为 **2** 时, 意味着卷积核在输入图像上每次移动 2 个像素, 这不仅能提取边缘特征, 还能实现图像的下采样 (降采样), 使输出特征图的尺寸减半。

下面通过一个具体的数值计算案例, 带你一步步推导带步长 2 的 Sobel 卷积计算过程。

1. 基础定义

(1) Sobel 卷积核 (3×3)

Sobel 算子包含水平方向 (G_x) 和垂直方向 (G_y) 两个卷积核:

- 水平梯度核 (G_x) (检测垂直边缘):

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

- 垂直梯度核 (G_y) (检测水平边缘):

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

(2) 步长与输出尺寸公式

- 步长 $S = 2$
- 卷积核大小 $K = 3$
- 若输入图像尺寸为 $W \times H$, 无填充 (Padding = 0), 输出尺寸计算公式为:

$$\text{Output Size} = \left\lfloor \frac{W - K}{S} \right\rfloor + 1$$

2. 计算案例

步骤一: 构造输入图像

假设我们有一张 7×7 的灰度图像 (左侧较暗, 右侧较亮, 中间有一个明显的垂直边缘):

$$\text{Input } I = \begin{bmatrix} 10 & 10 & 10 & 100 & 100 & 100 & 100 \\ 10 & 10 & 10 & 100 & 100 & 100 & 100 \\ 10 & 10 & 10 & 100 & 100 & 100 & 100 \\ 10 & 10 & 10 & 100 & 100 & 100 & 100 \\ 10 & 10 & 10 & 100 & 100 & 100 & 100 \\ 10 & 10 & 10 & 100 & 100 & 100 & 100 \\ 10 & 10 & 10 & 100 & 100 & 100 & 100 \end{bmatrix}$$

代入公式: $\lfloor (7 - 3)/2 \rfloor + 1 = 3$, 最终输出的特征图尺寸为 3×3 。

步骤二：计算水平梯度 (G_x)

由于输入图像在垂直方向上完全一致，垂直梯度 G_y 的计算结果全为 0。我们重点来看水平梯度 G_x 的计算。

卷积核每次在水平和垂直方向移动 2 步：

1. 输出位置 (0, 0)：对应输入图像左上角以 (0, 0) 为起点的 3×3 区域：

$$\begin{bmatrix} 10 & 10 & 10 \\ 10 & 10 & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

与 G_x 逐元素相乘并求和：

$$(-1 \times 10 + 0 \times 10 + 1 \times 10) + (-2 \times 10 + 0 \times 10 + 2 \times 10) + (-1 \times 10 + 0 \times 10 + 1 \times 10) = 0$$

2. 输出位置 (0, 1)：由于步长为 2，输入窗口向右滑动 2 列，起始列索引为 $0 + 2 = 2$ （即截取第 2 列到第 4 列）：

$$\begin{bmatrix} 10 & 10 & 100 \\ 10 & 10 & 100 \\ 10 & 10 & 100 \end{bmatrix}$$

与 G_x 相乘并求和：

- 第 1 行： $-1 \times 10 + 0 \times 10 + 1 \times 100 = 90$
- 第 2 行： $-2 \times 10 + 0 \times 10 + 2 \times 100 = 180$
- 第 3 行： $-1 \times 10 + 0 \times 10 + 1 \times 100 = 90$
- 总和 = $90 + 180 + 90 = 360$

3. 输出位置 (0, 2)：输入窗口起始列索引为 $2 + 2 = 4$ （截取第 4 列到第 6 列）：

$$\begin{bmatrix} 100 & 100 & 100 \\ 100 & 100 & 100 \\ 100 & 100 & 100 \end{bmatrix}$$

与 G_x 相乘并求和：

$$(-1 \times 100 + 0 \times 100 + 1 \times 100) \times 3 = 0$$

同理，由于整个图像上下结构对称，第 1 行和第 2 行的计算结果完全相同。

3. 最终计算结果

经过步长为 2 的 Sobel 卷积后，得到的输出矩阵为：

- 水平边缘响应 (O_x)：

$$\begin{bmatrix} 0 & 360 & 0 \\ 0 & 360 & 0 \\ 0 & 360 & 0 \end{bmatrix}$$

- 垂直边缘响应 (O_y)（因为上下无变化，全为 0）：

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

4. 核心特点总结

1. **降采样效果**: 输入尺寸由 7×7 缩小为 3×3 , 起到了类似池化层 (Pooling) 的空间压缩作用。
2. **边缘定位稀疏化**: 由于步长为 2, 跳过了部分像素, 中间列的强烈灰度跳变 (10 到 100 的突变) 被精确捕捉在输出矩阵的第二列 (值为 360)。
3. **计算效率**: 步长大于 1 时, 卷积核需要覆盖的有效滑动位置减少, 从而降低了后续特征处理的计算开销。

User prompt: 用 python 代码展示

Response: 下面是用 Python (基于 NumPy 库) 实现的带步长 (Stride = 2) 的 Sobel 卷积计算代码。代码中手写了一个简单的二维卷积函数, 方便你清晰地看到步长是如何控制窗口滑动的。

```
import numpy as np

# 1. 定义 7x7 的输入图像
input_image = np.array([
    [10, 10, 10, 100, 100, 100, 100],
    [10, 10, 10, 100, 100, 100, 100],
    [10, 10, 10, 100, 100, 100, 100],
    [10, 10, 10, 100, 100, 100, 100],
    [10, 10, 10, 100, 100, 100, 100],
    [10, 10, 10, 100, 100, 100, 100],
    [10, 10, 10, 100, 100, 100, 100]
], dtype=np.float32)

# 2. 定义 Sobel 卷积核 (Gx 和 Gy)
Gx = np.array([
    [-1, 0, 1],
    [-2, 0, 2],
    [-1, 0, 1]
], dtype=np.float32)

Gy = np.array([
    [-1, -2, -1],
    [ 0,  0,  0],
    [ 1,  2,  1]
], dtype=np.float32)

# 3. 实现自定义的带步长 (Stride) 二维卷积函数
def conv2d_stride(image, kernel, stride=2):
    h, w = image.shape
    kh, kw = kernel.shape

    # 计算无填充 (Padding=0) 时的输出尺寸
    out_h = (h - kh) // stride + 1
    out_w = (w - kw) // stride + 1

    output = np.zeros((out_h, out_w), dtype=np.float32)

    for i in range(out_h):
        for j in range(out_w):
            # 根据步长计算在输入图像中的起始位置
            r_start = i * stride
            c_start = j * stride
```

```
# 提取 3x3 局部窗口
window = image[r_start:r_start+kh, c_start:c_start+kw]

# 点乘并求和
output[i, j] = np.sum(window * kernel)

return output

# 4. 执行计算 (设置 stride=2)
output_gx = conv2d_stride(input_image, Gx, stride=2)
output_gy = conv2d_stride(input_image, Gy, stride=2)

print("水平梯度 Gx 输出 (3x3):")
print(output_gx)

print("\n垂直梯度 Gy 输出 (3x3):")
print(output_gy)
```

运行输出结果

水平梯度 Gx 输出 (3x3):

```
[[ 0. 360.  0.]
 [ 0. 360.  0.]
 [ 0. 360.  0.]
```

垂直梯度 Gy 输出 (3x3):

```
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]
```

你可以直接复制这段代码在本地运行，它通过 stride=2 实现了矩阵的空间降维，并且精准锁定了中间列的垂直边缘突变值 360。
